

Reporte generado por: Christopher Herrera

Análisis y correcciones: Christopher Herrera

Agente: Codex

Destino: Cinthia Robles

Capa: Frontend (Angular)

Objetivo: Poder localizar los componentes núcleo de Angular utilizados durante el proyecto para poder traspasar el conocimiento al resto del grupo y poder facilitar su comprensión y dudas a nivel de exposición.

Documentación de Componentes Angular

Climate Monitoring System

Repositorio: Climate_Frontend

Guía para personas que empiezan en sistemas

Copia analizada: rama test

9 de septiembre de 2026

Índice de secciones

1. Resumen de arquitectura
2. Services
3. Guards
4. Interceptors
5. Routing
6. Authentication
7. Features
8. Shared
9. Models e Interfaces
10. Pipes
11. Forms
12. Stores y gestión de estado
13. Components — páginas principales

Documentación de Componentes Angular

Climate Monitoring System

Repositorio de referencia: Climate_Frontend

Fecha de análisis: 9 de septiembre de 2026

1. Resumen de arquitectura

Árbol real de directorios y archivos TypeScript de producción; se omiten las plantillas, estilos y pruebas para facilitar la lectura.

```
src/
├── app/
│   ├── core/
│   │   ├── auth/
│   │   │   ├── auth-api.service.ts
│   │   │   ├── auth-session.model.ts
│   │   │   ├── auth.service.ts
│   │   │   ├── auth.store.ts
│   │   │   └── token-storage.service.ts
│   │   ├── config/
│   │   │   ├── app-config.model.ts
│   │   │   └── app-config.token.ts
│   │   ├── guards/
│   │   │   ├── auth.guard.ts
│   │   │   ├── guest.guard.ts
│   │   │   ├── pending-changes.guard.ts
│   │   │   └── role.guard.ts
│   │   ├── http/
│   │   │   ├── api-error.model.ts
│   │   │   └── problem-details.service.ts
│   │   ├── interceptors/
│   │   │   ├── api-error.interceptor.ts
│   │   │   └── auth.interceptor.ts
│   │   ├── layout/
│   │   │   ├── app-shell/
│   │   │   │   ├── app-shell.ts
│   │   │   ├── core-welcome-page/
│   │   │   │   └── core-welcome-page.ts
│   │   │   ├── feature-placeholder-page/
│   │   │   │   └── feature-placeholder-page.ts
│   │   │   ├── forbidden-page/
│   │   │   │   └── forbidden-page.ts
│   │   │   ├── login-placeholder-page/
│   │   │   ├── nav-icon/
│   │   │   │   └── nav-icon.ts
│   │   │   ├── sidebar/
│   │   │   │   └── sidebar.ts
│   │   │   ├── topbar/
│   │   │   │   └── topbar.ts
│   │   │   └── navigation.model.ts
│   │   ├── notifications/
│   │   │   └── toast.service.ts
│   │   ├── realtime/
│   │   │   └── realtime.service.ts
│   │   └── features/
│   │       ├── alerts/
│   │       │   ├── components/
│   │       │   │   └── alert-card/
│   │       │   │       └── alert-card.ts
│   │       │   ├── models/
│   │       │   │   └── alert.models.ts
│   │       │   ├── pages/
│   │       │   │   ├── alert-detail-page/
│   │       │   │   │   └── alert-detail-page.ts
│   │       │   │   ├── alerts-page/
│   │       │   │   │   └── alerts-page.ts
│   │       │   └── services/
│   │       │       └── alerts-api.service.ts
│   │       ├── audit/
│   │       │   ├── models/
│   │       │   │   └── audit.models.ts
│   │       │   └── pages/
```

```

├── audit-detail-page/
│   └── audit-detail-page.ts
├── audit-page/
│   └── audit-page.ts
├── services/
│   └── audit-api.service.ts
├── auth/
│   └── pages/
│       └── login-page/
│           └── login-page.ts
├── communities/
│   ├── models/
│   │   └── community.models.ts
│   ├── pages/
│   │   ├── communities-page/
│   │   │   └── communities-page.ts
│   │   ├── community-detail-page/
│   │   │   └── community-detail-page.ts
│   │   └── community-form-page/
│   │       └── community-form-page.ts
│   └── services/
│       └── communities-api.service.ts
├── dashboard/
│   ├── components/
│   │   ├── active-alerts-panel/
│   │   │   └── active-alerts-panel.ts
│   │   └── climate-metric-card/
│   │       └── climate-metric-card.ts
│   ├── pages/
│   │   └── dashboard-page/
│   │       └── dashboard-page.ts
│   └── store/
│       └── dashboard.store.ts
├── events/
│   ├── models/
│   │   └── event.models.ts
│   ├── pages/
│   │   ├── event-detail-page/
│   │   │   └── event-detail-page.ts
│   │   └── events-page/
│   │       └── events-page.ts
│   └── services/
│       └── events-api.service.ts
├── monitoring/
│   ├── components/
│   │   ├── sensor-chart/
│   │   │   └── sensor-chart.ts
│   │   └── simulation-control/
│   │       └── simulation-control.ts
│   ├── models/
│   │   └── monitoring.models.ts
│   ├── pages/
│   │   └── monitoring-page/
│   │       └── monitoring-page.ts
│   ├── services/
│   │   └── monitoring-api.service.ts
│   └── store/
│       └── monitoring.store.ts
├── sensors/
│   ├── models/
│   │   └── sensor.models.ts
│   ├── pages/
│   │   ├── sensor-detail-page/
│   │   │   └── sensor-detail-page.ts
│   │   ├── sensor-form-page/
│   │   │   └── sensor-form-page.ts
│   │   └── sensors-page/
│   │       └── sensors-page.ts
│   └── services/
│       └── sensors-api.service.ts
├── users/
│   ├── models/
│   │   └── user.models.ts
│   ├── pages/
│   │   ├── user-detail-page/
│   │   │   └── user-detail-page.ts
│   │   ├── user-edit-page/
│   │   │   └── user-edit-page.ts

```

```

|
|
|   └─ users-page/
|       └─ users-page.ts
|   └─ services/
|       └─ users-api.service.ts
|
|   └─ shared/
|       └─ components/
|           └─ toast-container/
|               └─ toast-container.ts
|       └─ models/
|           └─ api/
|               └─ auth.models.ts
|               └─ problem-details.model.ts
|       └─ testing/
|
|   └─ app.config.ts
|   └─ app.routes.ts
|   └─ app.ts
|
|   └─ environments/
|       └─ environment.production.ts
|       └─ environment.ts
|
|   └─ main.ts

```

2. Services

Un servicio realiza una tarea para otras partes del programa. Los servicios de comunicación envían datos al servidor (backend) desde el frontend y reciben su respuesta (response). Otros servicios trabajan dentro del navegador, por ejemplo para guardar la sesión o mostrar avisos. En esos casos no hay una solicitud HTTP directa.

Cómo leer las tablas: Body request muestra un ejemplo JSON de los datos enviados y response muestra un ejemplo JSON de la respuesta. Los valores son ficticios e ilustrativos; no son credenciales reales ni resultados de una consulta ejecutada. Los listados usan arreglos JSON y los detalles usan objetos JSON. TOKEN_DE_EJEMPLO representa el token que devolvería el servidor. Los valores del estado de simulación ilustran la estructura y dependen de la operación realizada. Los campos opcionales se incluyen como ejemplo y pueden omitirse. GET no envía body: sus filtros van en la URL. «Sin body» significa que no se envía o recibe contenido; no equivale a un objeto JSON vacío. 200 OK indica éxito con respuesta, 201 Created indica creación y 204 No Content indica éxito sin cuerpo de respuesta.

Origen de los códigos: Se verificaron los controladores AuthController, UsersController, SensorsController, CommunitiesController, MonitoringController, AlertsController, EventsController y AuditController, junto con sus ResultExtensions, en backend/src/Services. La configuración backend/src/Gateway/Climate.Gateway/appsettings.json relaciona las direcciones /api/ usadas por el frontend con /api/v1/ del backend. Son códigos esperados según el código local, no resultados de solicitudes ejecutadas. Los campos de request y response corresponden a los modelos TypeScript del frontend descritos en la sección de modelos.

AuthApiService

Categoría: Service

Descripción: Sirve para enviar datos al servidor (backend) desde el frontend: envía el usuario y la contraseña para iniciar sesión y recibe la respuesta (response) con la información de acceso.

Ruta del archivo:

src/app/core/auth/auth-api.service.ts

Utilizado por: src/app/core/auth/auth.service.ts

Dependencias principales: HttpClient, APP_CONFIG

Métodos principales: login, register

Verbo	URL construida	Body request (datos enviados)	Response (respuesta)	HTTP esperado (éxito)
POST	{apiUrl}/api/auth/login	{ "login": "usuario@example.com", "	{ "accessToken": "TOKEN_DE_EJEMPLO", "expiresAt":	200 OK

Verbo	URL construida	Body request (datos enviados)	Response (respuesta)	HTTP esperado (éxito)
		<pre> "password": "ClaveDeEjemplo123!" } </pre>	<pre> "2026-09- 09T13:00:00Z", "user": { "id": "11111111- 1111-4111-8111- 111111111111", "username": "usuario_ejemplo", "email": "usuario@example.com", "role": "Operator", "isActive": true, "createdAt": "2026-09- 09T12:00:00Z", "updatedAt": "2026-09- 09T12:00:00Z" } } </pre>	
POST	{apiUrl}/api/auth/register	<pre> { "username": "usuario_ejemplo", "email": "usuario@example.com", "password": "ClaveDeEjemplo123!" } </pre>	<pre> { "id": "11111111- 1111-4111-8111- 111111111111", "username": "usuario_ejemplo", "email": "usuario@example.com", "role": "Operator", "isActive": true, "createdAt": "2026-09- 09T12:00:00Z", "updatedAt": "2026-09- 09T12:00:00Z" } </pre>	201 Created

AuthService

Categoría: Service

Descripción: Coordina el envío de los datos de acceso desde el frontend al servidor (backend) mediante AuthApiService. Al recibir la respuesta (response) de acceso correcto, guarda la sesión. También permite cerrar sesión y volver a /login.

Ruta del archivo:

src/app/core/auth/auth.service.ts

Utilizado por: src/app/core/layout/sidebar/sidebar.ts; src/app/core/layout/topbar/topbar.ts;
src/app/features/auth/pages/login-page/login-page.ts

Dependencias principales: AuthApiService, AuthStore, Router, RealtimeService

Métodos principales: login, logout

No realiza llamadas HTTP directas; su colaboración está descrita arriba.

TokenStorageService

Categoría: Service

Descripción: Sirve para guardar, recuperar o borrar la sesión en el navegador del usuario. Trabaja en el frontend y no envía datos al servidor (backend).

Ruta del archivo:

`src/app/core/auth/token-storage.service.ts`

Utilizado por: `src/app/core/auth/auth.store.ts`

Dependencias principales: PLATFORM_ID

Métodos principales: read, write, clear

No realiza llamadas HTTP directas; su colaboración está descrita arriba.

ProblemDetailsService

Categoría: Service

Descripción: Sirve para convertir un error recibido del servidor (backend) en un mensaje que el frontend pueda mostrar. Procesa la respuesta (response) de error; no envía solicitudes al servidor.

Ruta del archivo:

`src/app/core/http/problem-details.service.ts`

Utilizado por: `src/app/core/interceptors/api-error.interceptor.ts`

Dependencias principales: No declarados.

Métodos principales: fromHttpError

No realiza llamadas HTTP directas; su colaboración está descrita arriba.

ToastService

Categoría: Service

Descripción: Sirve para mostrar avisos breves en el frontend, como «Sensor creado», y retirarlos después de unos segundos o cuando se cierran. No envía datos al servidor (backend).

Ruta del archivo:

`src/app/core/notifications/toast.service.ts`

Utilizado por: `src/app/core/interceptors/api-error.interceptor.ts`; `src/app/features/alerts/pages/alert-detail-page/alert-detail-page.ts`; `src/app/features/alerts/pages/alerts-page/alerts-page.ts`; `src/app/features/communities/pages/community-form-page/community-form-page.ts`; `src/app/features/monitoring/store/monitoring.store.ts`; `src/app/features/sensors/pages/sensor-form-page/sensor-form-page.ts`; `src/app/features/sensors/pages/sensors-page/sensors-page.ts`; `src/app/features/users/pages/user-edit-page/user-edit-page.ts`; `src/app/features/users/pages/users-page/users-page.ts`; `src/app/shared/components/toast-container/toast-container.ts`

Dependencias principales: No declarados.

Métodos principales: show, dismiss

No realiza llamadas HTTP directas; su colaboración está descrita arriba.

RealtimeService

Categoría: Service

Descripción: Sirve para enviar datos al servidor (backend) desde el frontend para preparar la conexión de actualizaciones en vivo. Recibe la respuesta (response) que permite abrir esa conexión y después recibe lecturas y avisos sin recargar la página.

Ruta del archivo:

src/app/core/realtime/realtime.service.ts

Utilizado por: src/app/core/auth/auth.service.ts; src/app/core/layout/app-shell/app-shell.ts;
src/app/features/alerts/pages/alerts-page/alerts-page.ts; src/app/features/dashboard/store/dashboard.store.ts;
src/app/features/monitoring/store/monitoring.store.ts

Dependencias principales: APP_CONFIG, AuthStore

Métodos principales: start, stop

Conexión externa: POST {hubUrl}/negotiate?negotiateVersion=1 con fetch y WebSocket al hub. Consulte Realtime; este transporte no usa HttpClient.

AlertsApiService

Categoría: Service

Descripción: Sirve para enviar datos al servidor (backend) desde el frontend para buscar alertas o marcar una como resuelta. Recibe la respuesta (response) con las alertas solicitadas o la confirmación de la operación.

Ruta del archivo:

src/app/features/alerts/services/alerts-api.service.ts

Utilizado por: src/app/features/alerts/pages/alert-detail-page/alert-detail-page.ts;
src/app/features/alerts/pages/alerts-page/alerts-page.ts; src/app/features/dashboard/store/dashboard.store.ts;
src/app/features/monitoring/store/monitoring.store.ts

Dependencias principales: HttpClient, APP_CONFIG

Métodos principales: getAll, getById, resolve

Verbo	URL construida	Body request (datos enviados)	Response (respuesta)	HTTP esperado (éxito)
GET	{apiUrl}/api/alerts	Sin body. Filtros opcionales en la URL: riskType, alertLevel, sensorId, communityId, isActive.	[{ "id": "55555555-5555-4555-8555-555555555555", "sensorId": "22222222-2222-4222-8222-222222222222", "communityId": "33333333-3333-4333-8333-333333333333", "alertType": "Drought", "level": "Yellow", "title": "Alerta de ejemplo", "description": "Datos ilustrativos de una alerta", "sensorValue": 35.0, "thresholdValue": 30.0, "generatedAt": "2026-09-	200 OK

Verbo	URL construida	Body request (datos enviados)	Response (respuesta)	HTTP esperado (éxito)
GET	{apiUrl}/api/alerts/\${id}	Sin body.	<pre> 09T12:00:00Z", "isActive": true, "resolvedAt": null }] </pre>	200 OK
PATCH	{apiUrl}/api/alerts/\${id}/resolve	Sin body (el frontend pasa null).	<pre> { "id": "55555555-5555-4555-8555-555555555555", "sensorId": "22222222-2222-4222-8222-222222222222", "communityId": "33333333-3333-4333-8333-333333333333", "alertType": "Drought", "level": "Yellow", "title": "Alerta de ejemplo", "description": "Datos ilustrativos de una alerta", "sensorValue": 35.0, "thresholdValue": 30.0, "generatedAt": "2026-09-09T12:00:00Z", "isActive": true, "resolvedAt": null } </pre>	204 No Content

AuditApiService

Categoría: Service

Descripción: Sirve para enviar datos al servidor (backend) desde el frontend para buscar acciones registradas por usuario, acción, recurso o fechas. Recibe la respuesta (response) con el historial solicitado o un registro específico.

Ruta del archivo:

src/app/features/audit/services/audit-api.service.ts

Utilizado por: src/app/features/audit/pages/audit-detail-page/audit-detail-page.ts;
src/app/features/audit/pages/audit-page/audit-page.ts

Dependencias principales: HttpClient, APP_CONFIG

Métodos principales: getAll, getById

Verbo	URL construida	Body request (datos enviados)	Response (respuesta)	HTTP esperado (éxito)
GET	{apiUrl}/api/audit	Sin body. Filtros opcionales en la URL: userId, action, resource, from, to.	<pre> [{ "id": "66666666-6666-4666-8666-666666666666", "userId": "11111111-1111-4111-8111-111111111111", "userName": "usuario_ejemplo", "action": "UpdateSensor", "resource": </pre>	200 OK

Verbo	URL construida	Body request (datos enviados)	Response (respuesta)	HTTP esperado (éxito)
GET	{apiUrl}/api/audit/{id}	Sin body.	<pre> { "Sensor", "resourceId": "22222222-2222-4222-8222-222222222222", "description": "Actualización de un sensor de ejemplo", "ipAddress": "192.0.2.10", "timestamp": "2026-09-09T12:00:00Z" } </pre>	200 OK

CommunitiesApiService

Categoría: Service

Descripción: Sirve para enviar datos al servidor (backend) desde el frontend para consultar, crear o modificar comunidades. Recibe la respuesta (response) con la lista o los datos de la comunidad.

Ruta del archivo:

src/app/features/communities/services/communities-api.service.ts

Utilizado por: src/app/features/alerts/pages/alert-detail-page/alert-detail-page.ts;
 src/app/features/alerts/pages/alerts-page/alerts-page.ts;
 src/app/features/communities/pages/communities-page/communities-page.ts;
 src/app/features/communities/pages/community-detail-page/community-detail-page.ts;
 src/app/features/communities/pages/community-form-page/community-form-page.ts;
 src/app/features/events/pages/event-detail-page/event-detail-page.ts;
 src/app/features/events/pages/events-page/events-page.ts; src/app/features/sensors/pages/sensor-form-page/sensor-form-page.ts

Dependencias principales: HttpClient, APP_CONFIG

Métodos principales: getAll, getById, create, update

Verbo	URL construida	Body request (datos enviados)	Response (respuesta)	HTTP esperado (éxito)
GET	{apiUrl}/api/communities	Sin body.	[{	200 OK

Verbo	URL construida	Body request (datos enviados)	Response (respuesta)	HTTP esperado (éxito)
GET	{apiUrl}/api/ communities/\${id}	Sin body.	<pre> "id": "333333333- 3333-4333-8333- 333333333333", "name": "Comunidad de ejemplo", "description": "Comunidad para ilustrar los datos", "latitude": 14.64, "longitude": -90.51, "isActive": true, "createdAt": "2026-09- 09T12:00:00Z" }] </pre>	200 OK
			<pre> { "id": "333333333- 3333-4333-8333- 333333333333", "name": "Comunidad de ejemplo", "description": "Comunidad para ilustrar los datos", "latitude": 14.64, "longitude": -90.51, "isActive": true, "createdAt": "2026-09- 09T12:00:00Z" } </pre>	
POST	{apiUrl}/api/ communities	<pre> { "name": "Comunidad de ejemplo", "description": "Comunidad para ilustrar los datos", "latitude": 14.64, "longitude": -90.51 } </pre>	<pre> { "id": "333333333- 3333-4333-8333- 333333333333", "name": "Comunidad de ejemplo", "description": "Comunidad para ilustrar los datos", "latitude": 14.64, "longitude": -90.51, "isActive": true, "createdAt": "2026-09- 09T12:00:00Z" } </pre>	201 Created
PUT	{apiUrl}/api/ communities/\${id}	<pre> { "name": "Comunidad de ejemplo", "description": "Comunidad para ilustrar los datos", "latitude": 14.64, "longitude": -90.51, "isActive": true } </pre>	<pre> { "id": "333333333- 3333-4333-8333- 333333333333", "name": "Comunidad de ejemplo", "description": "Comunidad para ilustrar los datos", "latitude": 14.64, "longitude": -90.51, "isActive": true, "createdAt": "2026-09- 09T12:00:00Z" } </pre>	200 OK

EventsApiService

Categoría: Service

Descripción: Sirve para enviar datos al servidor (backend) desde el frontend para buscar eventos climáticos por riesgo, nivel, sensor, comunidad o fechas. Recibe la respuesta (response) con los eventos o el detalle solicitado.

Ruta del archivo:

src/app/features/events/services/events-api.service.ts

Utilizado por: src/app/features/events/pages/event-detail-page/event-detail-page.ts;
src/app/features/events/pages/events-page/events-page.ts

Dependencias principales: HttpClient, APP_CONFIG

Métodos principales: getAll, getById

Verbo	URL construida	Body request (datos enviados)	Response (respuesta)	HTTP esperado (éxito)
GET	{apiUrl}/api/events	Sin body. Filtros opcionales en la URL: riskType, alertLevel, sensorId, communityId, from, to.	[{ "id": "77777777-7777-4777-8777-777777777777", "alertId": "55555555-5555-4555-8555-555555555555", "sensorId": "22222222-2222-4222-8222-222222222222", "communityId": "33333333-3333-4333-8333-333333333333", "riskType": "Drought", "alertLevel": "Yellow", "description": "Evento de ejemplo", "occurredAt": "2026-09-09T12:00:00Z", "resolvedAt": null }]	200 OK
GET	{apiUrl}/api/events/{id}	Sin body.	{ "id": "77777777-7777-4777-8777-777777777777", "alertId": "55555555-5555-4555-8555-555555555555", "sensorId": "22222222-2222-4222-8222-222222222222", "communityId": "33333333-3333-4333-8333-333333333333", "riskType": "Drought", "alertLevel": "Yellow", "description": "Evento de ejemplo", "occurredAt": "2026-09-09T12:00:00Z", "resolvedAt": null }	200 OK

MonitoringApiService

Categoría: Service

Descripción: Sirve para enviar datos al servidor (backend) desde el frontend para consultar mediciones y gráficas, registrar una lectura o controlar la simulación. Recibe la respuesta (response) con las lecturas, la gráfica o el estado de la simulación.

Ruta del archivo:

src/app/features/monitoring/services/monitoring-api.service.ts

Utilizado por: src/app/features/dashboard/store/dashboard.store.ts;
src/app/features/monitoring/store/monitoring.store.ts

Dependencias principales: HttpClient, APP_CONFIG

Métodos principales: getCurrent, getSimulationStatus, getLatest, getHistory, getChart, createReading, startSimulation, stopSimulation, resetSimulation

Verbo	URL construida	Body request (datos enviados)	Response (respuesta)	HTTP esperado (éxito)
GET	{apiUrl}/api/monitoring/current	Sin body.	[{ "id": "44444444-4444-4444-8444-444444444444", "sensorId": "22222222-2222-4222-8222-222222222222", "communityId": "33333333-3333-4333-8333-333333333333", "sensorType": "Temperature", "value": 24.5, "unit": "°C", "recordedAt": "2026-09-09T12:00:00Z" }]	200 OK
GET	{apiUrl}/api/monitoring/simulation/status	Sin body.	{ "isRunning": true }	200 OK
GET	{apiUrl}/api/monitoring/sensors/{sensorId}/latest	Sin body.	{ "id": "44444444-4444-4444-8444-444444444444", "sensorId": "22222222-2222-4222-8222-222222222222", "communityId": "33333333-3333-4333-8333-333333333333", "sensorType": "Temperature", "value": 24.5, "unit": "°C", "recordedAt": "2026-09-09T12:00:00Z" }	200 OK
GET	{apiUrl}/api/monitoring/sensors/{sensorId}/history	Sin body. Filtros opcionales en la URL: communityId, from, to.	[{ "id": "44444444-4444-4444-8444-444444444444", "sensorId": "22222222-2222-4222-8222-222222222222", "communityId": "33333333-3333-4333-8333-333333333333", "sensorType": "Temperature", "value": 24.5, "unit": "°C", "recordedAt": "2026-09-09T12:00:00Z" }]	200 OK

Verbo	URL construida	Body request (datos enviados)	Response (respuesta)	HTTP esperado (éxito)
			<pre> "unit": "°C", "recordedAt": "2026-09- 09T12:00:00Z" }] </pre>	
GET	{apiUrl}/api/monitoring/sensors/{sensorId}/chart	Sin body. Filtros opcionales en la URL: from, to, interval.	<pre> { "sensorId": "22222222-2222-4222- 8222-222222222222", "unit": "°C", "data": [{ "timestamp": "2026-09- 09T12:00:00Z", "value": 24.5 }] } </pre>	200 OK
POST	{apiUrl}/api/monitoring/readings	<pre> { "sensorId": "22222222-2222-4222- 8222-222222222222", "value": 24.5, "recordedAt": "2026-09- 09T12:00:00Z" } </pre>	<pre> { "id": "44444444- 4444-4444-8444- 444444444444", "sensorId": "22222222-2222-4222- 8222-222222222222", "communityId": "33333333-3333-4333- 8333-333333333333", "sensorType": "Temperature", "value": 24.5, "unit": "°C", "recordedAt": "2026-09- 09T12:00:00Z" } </pre>	201 Created
POST	{apiUrl}/api/monitoring/simulation/start	Sin body (el frontend pasa null).	<pre> { "isRunning": true } </pre>	200 OK
POST	{apiUrl}/api/monitoring/simulation/stop	Sin body (el frontend pasa null).	<pre> { "isRunning": true } </pre>	200 OK
POST	{apiUrl}/api/monitoring/simulation/reset	Sin body (el frontend pasa null).	<pre> { "isRunning": true } </pre>	200 OK

SensorsApiService

Categoría: Service

Descripción: Sirve para enviar datos al servidor (backend) desde el frontend para consultar, crear, modificar, eliminar, activar o desactivar sensores. Recibe la respuesta (response) con los datos solicitados o la confirmación de la operación.

Ruta del archivo:

src/app/features/sensors/services/sensors-api.service.ts

Utilizado por: src/app/features/alerts/pages/alert-detail-page/alert-detail-page.ts;
 src/app/features/alerts/pages/alerts-page/alerts-page.ts; src/app/features/dashboard/store/dashboard.store.ts;
 src/app/features/events/pages/event-detail-page/event-detail-page.ts;
 src/app/features/events/pages/events-page/events-page.ts; src/app/features/monitoring/store/monitoring.store.ts;
 src/app/features/sensors/pages/sensor-detail-page/sensor-detail-page.ts; src/app/features/sensors/pages/sensor-form-page/sensor-form-page.ts; src/app/features/sensors/pages/sensors-page/sensors-page.ts

Dependencias principales: HttpClient, APP_CONFIG

Métodos principales: getAll, getById, create, update, delete, activate, deactivate

Verbo	URL construida	Body request (datos enviados)	Response (respuesta)	HTTP esperado (éxito)
GET	{apiUrl}/api/sensors	Sin body.	[<pre>{ "id": "22222222-2222-4222-8222-222222222222", "name": "Sensor de temperatura", "code": "TEMP-001", "description": "Sensor de ejemplo", "type": "Temperature", "unit": "°C", "communityId": "33333333-3333-4333-8333-333333333333", "communityName": "Comunidad de ejemplo", "latitude": 14.64, "longitude": -90.51, "isActive": true, "createdAt": "2026-09-09T12:00:00Z", "updatedAt": "2026-09-09T12:00:00Z" }</pre>	200 OK
GET	{apiUrl}/api/sensors/{id}	Sin body.	{ <pre>"id": "22222222-2222-4222-8222-222222222222", "name": "Sensor de temperatura", "code": "TEMP-001", "description": "Sensor de ejemplo", "type": "Temperature", "unit": "°C", "communityId": "33333333-3333-4333-8333-333333333333", "communityName": "Comunidad de ejemplo", "latitude": 14.64, "longitude": -90.51, "isActive": true, "createdAt": "2026-09-09T12:00:00Z", "updatedAt": "2026-09-09T12:00:00Z"</pre>	200 OK
POST	{apiUrl}/api/sensors	{ <pre>"name": "Sensor de temperatura", "code": "TEMP-001", "description": "Sensor de ejemplo", "type": "Temperature", "unit": "°C", "communityId": "33333333-3333-4333-</pre>	{ <pre>"id": "22222222-2222-4222-8222-222222222222", "name": "Sensor de temperatura", "code": "TEMP-001", "description": "Sensor de ejemplo", "type": "Temperature",</pre>	201 Created

Verbo	URL construida	Body request (datos enviados)	Response (respuesta)	HTTP esperado (éxito)
		<pre>8333-333333333333", "latitude": 14.64, "longitude": -90.51 }</pre>	<pre>"unit": "°C", "communityId": "33333333-3333-4333- 8333-333333333333", "communityName": "Comunidad de ejemplo", "latitude": 14.64, "longitude": -90.51, "isActive": true, "createdAt": "2026-09- 09T12:00:00Z", "updatedAt": "2026-09- 09T12:00:00Z" }</pre>	
PUT	{apiUrl}/api/sensors/\${id}	<pre>{ "name": "Sensor de temperatura", "code": "TEMP-001", "description": "Sensor de ejemplo", "type": "Temperature", "unit": "°C", "communityId": "33333333-3333-4333- 8333-333333333333", "latitude": 14.64, "longitude": -90.51 }</pre>	<pre>{ "id": "22222222- 2222-4222-8222- 222222222222", "name": "Sensor de temperatura", "code": "TEMP-001", "description": "Sensor de ejemplo", "type": "Temperature", "unit": "°C", "communityId": "33333333-3333-4333- 8333-333333333333", "communityName": "Comunidad de ejemplo", "latitude": 14.64, "longitude": -90.51, "isActive": true, "createdAt": "2026-09- 09T12:00:00Z", "updatedAt": "2026-09- 09T12:00:00Z" }</pre>	200 OK
DELETE	{apiUrl}/api/sensors/\${id}	Sin body.	Sin body de respuesta (void).	204 No Content
PATCH	{apiUrl}/api/sensors/\${id}/activate	Sin body (el frontend pasa null).	Sin body de respuesta (void).	204 No Content
PATCH	{apiUrl}/api/sensors/\${id}/deactivate	Sin body (el frontend pasa null).	Sin body de respuesta (void).	204 No Content

UsersApiService

Categoría: Service

Descripción: Sirve para enviar datos al servidor (backend) desde el frontend para consultar cuentas, cambiar sus datos o activar y desactivar su acceso. Recibe la respuesta (response) con los usuarios, la cuenta actualizada o la confirmación del cambio.

Ruta del archivo:

src/app/features/users/services/users-api.service.ts

Utilizado por: src/app/features/audit/pages/audit-page/audit-page.ts; src/app/features/users/pages/user-detail-page/user-detail-page.ts; src/app/features/users/pages/user-edit-page/user-edit-page.ts; src/app/features/users/pages/users-page/users-page.ts

Dependencias principales: HttpClient, APP_CONFIG**Métodos principales:** getAll, getById, update, updateStatus

Verbo	URL construida	Body request (datos enviados)	Response (respuesta)	HTTP esperado (éxito)
GET	{apiUrl}/api/users	Sin body.	[{ "id": "11111111-1111-4111-8111-111111111111", "username": "usuario_ejemplo", "email": "usuario@example.com", "role": "Operator", "isActive": true, "createdAt": "2026-09-09T12:00:00Z", "updatedAt": "2026-09-09T12:00:00Z" }]	200 OK
GET	{apiUrl}/api/users/{id}	Sin body.	{ "id": "11111111-1111-4111-8111-111111111111", "username": "usuario_ejemplo", "email": "usuario@example.com", "role": "Operator", "isActive": true, "createdAt": "2026-09-09T12:00:00Z", "updatedAt": "2026-09-09T12:00:00Z" }	200 OK
PUT	{apiUrl}/api/users/{id}	{ "username": "usuario_ejemplo", "email": "usuario@example.com", "role": "Operator" }	{ "id": "11111111-1111-4111-8111-111111111111", "username": "usuario_ejemplo", "email": "usuario@example.com", "role": "Operator", "isActive": true, "createdAt": "2026-09-09T12:00:00Z", "updatedAt": "2026-09-09T12:00:00Z" }	200 OK
PATCH	{apiUrl}/api/users/{id}/status	{ "isActive": false }	Sin body de respuesta (void).	204 No Content

3. Guards

Un guard funciona como un control de paso entre páginas. Antes de entrar o salir, revisa una condición y decide si deja continuar o lleva a otra página. Por ejemplo, puede comprobar si la persona ya inició sesión o si tiene el permiso necesario.

authGuard

Categoría: Guard

Descripción: Sirve para comprobar si la persona tiene una sesión que el sistema considera válida antes de entrar a las páginas privadas. Si no la tiene, la lleva a /login y recuerda la página que quería abrir.

Ruta del archivo:

`src/app/core/guards/auth.guard.ts`

Utilizado por: `src/app/app.routes.ts`

Dependencias principales: AuthStore, Router

Rutas registradas: /

guestGuard

Categoría: Guard

Descripción: Sirve para comprobar si la persona ya inició sesión. Si ya lo hizo, la lleva a /dashboard para que no vuelva a la pantalla de acceso. Si todavía no inició sesión, le permite entrar a /login.

Ruta del archivo:

`src/app/core/guards/guest.guard.ts`

Utilizado por: `src/app/app.routes.ts`

Dependencias principales: AuthStore, Router

Rutas registradas: /login

pendingChangesGuard

Categoría: Guard

Descripción: Sirve para impedir que una persona salga de una página cuando esta indica que tiene cambios pendientes. Si no hay cambios, permite salir. Actualmente está definido, pero no se usa en ninguna ruta y no muestra una pregunta de confirmación.

Ruta del archivo:

`src/app/core/guards/pending-changes.guard.ts`

Utilizado por: Sin consumidor externo identificado en el código de producción; las referencias internas se indican en el propósito.

Dependencias principales: PendingChangesAware (contrato del componente).

Rutas registradas: Ninguna. No está activo en la configuración de rutas.

roleGuard

Categoría: Guard

Descripción: Sirve para revisar si el tipo de usuario tiene permiso para entrar a una página. Si tiene permiso, lo deja pasar; si no, lo lleva a /forbidden. Si la página no exige un tipo de usuario específico, permite el acceso.

Ruta del archivo:

`src/app/core/guards/role.guard.ts`

Utilizado por: `src/app/app.routes.ts`

Dependencias principales: AuthStore, Router

Rutas registradas: `/sensors/new` → 'Administrator', 'Operator'; `/sensors/:id/edit` → 'Administrator', 'Operator'; `/communities/new` → 'Administrator', 'Operator'; `/communities/:id/edit` → 'Administrator', 'Operator'; `/users` → 'Administrator'; `/users/:id/edit` → 'Administrator'; `/users/:id` → 'Administrator'; `/audit` → 'Administrator'; `/audit/:id` → 'Administrator'; `/system` → 'Administrator'

4. Interceptors

Un interceptor revisa una solicitud antes de enviarla al servidor o una respuesta cuando regresa. Se puede imaginar como un punto de revisión: uno agrega la identificación de la sesión y otro revisa los errores. Aquí se aplican a las solicitudes hechas con HttpClient, la herramienta de Angular para comunicarse con el servidor.

apiErrorInterceptor

Categoría: Interceptor

Descripción: Sirve para revisar los errores que devuelve el servidor y preparar su mensaje. Si recibe un error de acceso no válido y había una sesión guardada, la borra y lleva a `/login`. Si el error indica falta de permisos, muestra un aviso.

Ruta del archivo:

`src/app/core/interceptors/api-error.interceptor.ts`

Utilizado por: `src/app/app.config.ts`

Dependencias principales: AuthStore, ProblemDetailsService, Router, ToastService

authInterceptor

Categoría: Interceptor

Descripción: Sirve para acompañar las solicitudes al servidor de la aplicación con la identificación de la sesión. Solo la agrega cuando existe un token, que es el comprobante de acceso recibido al iniciar sesión, y la dirección corresponde al servidor configurado.

Ruta del archivo:

`src/app/core/interceptors/auth.interceptor.ts`

Utilizado por: `src/app/app.config.ts`

Dependencias principales: APP_CONFIG, AuthStore

5. Routing

El Router es el encargado de abrir una pantalla según la dirección del navegador. Por ejemplo, `/login` abre el inicio de sesión y `/dashboard` abre el resumen del sistema. El archivo `app.routes.ts` reúne estas decisiones y sus controles de acceso. Algunas páginas se cargan solo cuando se necesitan; eso se llama carga diferida o lazy loading.

Ruta	Componente / Feature	Guard	Tipo	Descripción
<code>/login</code>	LoginPage	guestGuard	Pública para invitados; lazy	
<code>/</code>	AppShell	authGuard	Protegida; directa / redirección	
<code>/dashboard</code>	DashboardPage	authGuard en padre; sin guard propio	Protegida; lazy	Dashboard
<code>/monitoring</code>	MonitoringPage	authGuard en padre; sin guard propio	Protegida; lazy	Monitoreo en tiempo real
<code>/sensors</code>	SensorsPage	authGuard en padre; sin guard propio	Protegida; lazy	Sensores
<code>/sensors/new</code>	SensorFormPage	authGuard en padre; roleGuard	Protegida; lazy	Nuevo sensor Roles: 'Administrator', 'Operator'

Ruta	Componente / Feature	Guard	Tipo	Descripción
/sensors/:id/edit	SensorFormPage	authGuard en padre; roleGuard	Protegida; lazy	Editar sensor Roles: 'Administrator', 'Operator'
/sensors/:id	SensorDetailPage	authGuard en padre; sin guard propio	Protegida; lazy	Detalle del sensor
/communities	CommunitiesPage	authGuard en padre; sin guard propio	Protegida; lazy	Comunidades
/communities/new	CommunityFormPage	authGuard en padre; roleGuard	Protegida; lazy	Nueva comunidad Roles: 'Administrator', 'Operator'
/communities/:id/edit	CommunityFormPage	authGuard en padre; roleGuard	Protegida; lazy	Editar comunidad Roles: 'Administrator', 'Operator'
/communities/:id	CommunityDetailPage	authGuard en padre; sin guard propio	Protegida; lazy	Detalle de comunidad
/alerts	AlertsPage	authGuard en padre; sin guard propio	Protegida; lazy	Alertas
/alerts/:id	AlertDetailPage	authGuard en padre; sin guard propio	Protegida; lazy	Detalle de alerta
/events	EventsPage	authGuard en padre; sin guard propio	Protegida; lazy	Historial de eventos
/events/:id	EventDetailPage	authGuard en padre; sin guard propio	Protegida; lazy	Detalle del evento
/users	UsersPage	authGuard en padre; roleGuard	Protegida; lazy	Usuarios Roles: 'Administrator'
/users/:id/edit	UserEditPage	authGuard en padre; roleGuard	Protegida; lazy	Editar usuario Roles: 'Administrator'
/users/:id	UserDetailPage	authGuard en padre; roleGuard	Protegida; lazy	Detalle del usuario Roles: 'Administrator'
/audit	AuditPage	authGuard en padre; roleGuard	Protegida; lazy	Auditoría Roles: 'Administrator'
/audit/:id	AuditDetailPage	authGuard en padre; roleGuard	Protegida; lazy	Detalle de auditoría Roles: 'Administrator'
/system	FeaturePlaceholderPage	authGuard en padre; roleGuard	Protegida; directa / redirección	Configuración del sistema Roles: 'Administrator'
/forbidden	ForbiddenPage	authGuard en padre; sin guard propio	Protegida; directa / redirección	Acceso restringido
/	Redirección a dashboard	authGuard en padre; sin guard propio	Protegida; directa / redirección	Redirige a dashboard (pathMatch: full)
/**	Redirección a dashboard	sin guard propio	directa / redirección	Redirige a dashboard

Las dos filas / corresponden al shell padre y a su hijo vacío que redirige a dashboard. /** es el comodín final y también redirige a dashboard. /forbidden permanece dentro del shell autenticado. /system es una página placeholder para Administrator. CoreWelcomePage no tiene ruta. pendingChangesGuard no está asociado a canDeactivate.

6. Authentication

La autenticación consiste en comprobar quién está usando la aplicación. Al iniciar sesión correctamente, el servidor entrega un token, que funciona como comprobante de acceso. El frontend lo guarda junto con los datos de la cuenta y lo envía al realizar solicitudes. No comprueba por sí mismo la firma de ese token. La recuperación de una sesión revisa que tenga token, una fecha de vencimiento futura y un usuario activo.

Elemento	Propósito / relación con autenticación	Ruta	Consumidores
AuthApiService	Sirve para enviar al servidor los datos con los que una persona intenta iniciar sesión y recibir su respuesta.	src/app/core/auth/auth-api.service.ts	src/app/core/auth/auth.service.ts
AuthSession	Define la información que se guarda de una sesión: comprobante de acceso, fecha de vencimiento y datos de la persona conectada.	src/app/core/auth/auth-session.model.ts	src/app/core/auth/auth.store.ts; src/app/core/auth/token-storage.service.ts
AuthService	Sirve para coordinar el inicio y el cierre de sesión. Cuando el acceso es correcto, guarda la información de la sesión; al salir, la borra, detiene las actualizaciones en vivo y normalmente lleva a /login.	src/app/core/auth/auth.service.ts	src/app/core/layout/sidebar/sidebar.ts; src/app/core/layout/topbar/topbar.ts; src/app/features/auth/pages/login-page/login-page.ts
AuthStore	Sirve para mantener a mano la información de la persona que inició sesión y saber qué acciones puede realizar. También recupera	src/app/core/auth/auth.store.ts	src/app/app.ts; src/app/core/auth/auth.service.ts; src/app/core/guards/auth.guard.ts;

Elemento	Propósito / relación con autenticación	Ruta	Consumidores
	una sesión guardada si cumple las condiciones de acceso del sistema.		src/app/core/guards/guest.guard.ts; src/app/core/guards/role.guard.ts; src/app/core/interceptors/api-error.interceptor.ts; src/app/core/interceptors/auth.interceptor.ts; src/app/core/layout/sidebar/sidebar.ts; src/app/core/layout/topbar/topbar.ts; src/app/core/realtime/realtime.service.ts; src/app/features/alerts/pages/alert-detail-page/alert-detail-page.ts; src/app/features/alerts/pages/alerts-page/alerts-page.ts; src/app/features/communities/pages/communities-page/communities-page.ts; src/app/features/communities/pages/community-detail-page/community-detail-page.ts; src/app/features/monitoring/pages/monitoring-page/monitoring-page.ts; src/app/features/sensors/pages/sensor-detail-page/sensor-detail-page.ts; src/app/features/sensors/pages/sensors-page/sensors-page.ts; src/app/features/users/pages/user-edit-page/user-edit-page.ts; src/app/features/users/pages/users-page/users-page.ts
TokenStorageService	Sirve para guardar, recuperar o borrar la sesión en el navegador. Gracias a esto, la aplicación puede intentar recuperar el acceso cuando se vuelve a abrir o recargar la página.	src/app/core/auth/token-storage.service.ts	src/app/core/auth/auth.store.ts
SESSION_KEY	Sirve como nombre del espacio del navegador donde se guarda la sesión, para encontrarla o borrarla después.	src/app/core/auth/token-storage.service.ts	Sin consumidor externo identificado en el código de producción; las referencias internas se indican en el propósito.

Los contratos compartidos SystemRole, LoginRequest, LoginResponse, RegisterRequest y UserResponse están en src/app/shared/models/api/auth.models.ts. LoginPage → AuthService → AuthApiService → HttpClient; AuthService escribe AuthStore → TokenStorageService. App restaura la sesión; Sidebar y Topbar cierran sesión; guards e interceptores consultan AuthStore.

7. Features

Una feature es una sección del sistema que agrupa archivos relacionados con una tarea, como administrar sensores o consultar alertas. Se encontraron nueve secciones. Las carpetas pages, components, models, services y store organizan los archivos dentro de cada una.

alerts

Categoría: Feature

Descripción: Esta sección sirve para consultar, filtrar y resolver alertas climáticas.

Ruta de la carpeta:

src/app/features/alerts

Componentes: AlertCard, AlertDetailPage, AlertsPage

Servicios y stores: AlertsApiService

Modelos e interfaces locales: AlertLevel, RiskType, AlertResponse, AlertFilters

Rutas: /alerts, /alerts/:id

Dependencias relevantes: APP_CONFIG, ActivatedRoute, AlertsApiService, AuthStore, CommunitiesApiService, DestroyRef, HttpClient, RealtimeService, SensorsApiService, ToastService

Contratos externos utilizados: CommunityResponse, SensorResponse

audit

Categoría: Feature

Descripción: Esta sección sirve para consultar la bitácora y sus detalles con acceso administrativo.

Ruta de la carpeta:

src/app/features/audit

Componentes: AuditDetailPage, AuditPage

Servicios y stores: AuditApiService

Modelos e interfaces locales: AuditResponse, AuditFilters

Rutas: /audit, /audit/:id

Dependencias relevantes: APP_CONFIG, ActivatedRoute, AuditApiService, HttpClient, UsersApiService

Contratos externos utilizados: UserResponse

auth

Categoría: Feature

Descripción: Esta sección sirve para autenticar usuarios mediante la página de acceso.

Ruta de la carpeta:

src/app/features/auth

Componentes: LoginPage

Servicios y stores: No se identificaron.

Modelos e interfaces locales: No se identificaron.

Rutas: /login

Dependencias relevantes: ActivatedRoute, AuthService, DestroyRef, Router

Contratos externos utilizados: No declarados.

communities

Categoría: Feature

Descripción: Esta sección sirve para consultar, crear y editar comunidades geográficas.

Ruta de la carpeta:

`src/app/features/communities`

Componentes: CommunitiesPage, CommunityDetailPage, CommunityFormPage

Servicios y stores: CommunitiesApiService

Modelos e interfaces locales: CommunityResponse, CreateCommunityRequest, UpdateCommunityRequest

Rutas: /communities, /communities/new, /communities/:id/edit, /communities/:id

Dependencias relevantes: APP_CONFIG, ActivatedRoute, AuthStore, CommunitiesApiService, HttpClient, Router, ToastService

Contratos externos utilizados: No declarados.

dashboard

Categoría: Feature

Descripción: Esta sección sirve para resumir variables climáticas, sensores, simulación y alertas.

Ruta de la carpeta:

`src/app/features/dashboard`

Componentes: ActiveAlertsPanel, ClimateMetricCard, DashboardPage

Servicios y stores: DashboardStore

Modelos e interfaces locales: ClimateMetric

Rutas: /dashboard

Dependencias relevantes: AlertsApiService, DashboardStore, DestroyRef, MonitoringApiService, RealtimeService, SensorsApiService

Contratos externos utilizados: AlertResponse, SensorReadingResponse, SensorResponse, SensorType, SimulationStatusResponse

events

Categoría: Feature

Descripción: Esta sección sirve para consultar el historial de eventos y su contexto.

Ruta de la carpeta:

`src/app/features/events`

Componentes: EventDetailPage, EventsPage

Servicios y stores: EventsApiService

Modelos e interfaces locales: EventResponse, EventFilters

Rutas: /events, /events/:id

Dependencias relevantes: APP_CONFIG, ActivatedRoute, CommunitiesApiService, EventsApiService, HttpClient, SensorsApiService

Contratos externos utilizados: AlertLevel, CommunityResponse, RiskType, SensorResponse

monitoring

Categoría: Feature

Descripción: Esta sección sirve para consultar lecturas y gráficas y operar simulación con realtime y respaldo periódico.

Ruta de la carpeta:

src/app/features/monitoring

Componentes: SensorChart, SimulationControl, MonitoringPage

Servicios y stores: MonitoringApiService, MonitoringStore

Modelos e interfaces locales: SensorType, SensorReadingResponse, SimulationStatusResponse, ChartPoint, SensorChartResponse, ReadingHistoryFilters, ChartFilters, CreateReadingRequest

Rutas: /monitoring

Dependencias relevantes: APP_CONFIG, AlertsApiService, AuthStore, DOCUMENT, DestroyRef, HttpClient, MonitoringApiService, MonitoringStore, RealtimeService, SensorsApiService, ToastService

Contratos externos utilizados: AlertResponse, SensorResponse

sensors

Categoría: Feature

Descripción: Esta sección sirve para administrar inventario, estado y datos de sensores.

Ruta de la carpeta:

src/app/features/sensors

Componentes: SensorDetailPage, SensorFormPage, SensorsPage

Servicios y stores: SensorsApiService

Modelos e interfaces locales: SensorResponse, SensorRequest

Rutas: /sensors, /sensors/new, /sensors/:id/edit, /sensors/:id

Dependencias relevantes: APP_CONFIG, ActivatedRoute, AuthStore, CommunitiesApiService, HttpClient, Router, SensorsApiService, ToastService

Contratos externos utilizados: CommunityResponse, SensorType

users

Categoría: Feature

Descripción: Esta sección sirve para administrar identidad, rol y acceso de usuarios.

Ruta de la carpeta:

src/app/features/users

Componentes: UserDetailPage, UserEditPage, UsersPage

Servicios y stores: UsersApiService

Modelos e interfaces locales: UpdateUserRequest, UpdateUserStatusRequest

Rutas: /users, /users/:id/edit, /users/:id

Dependencias relevantes: APP_CONFIG, ActivatedRoute, AuthStore, HttpClient, Router, ToastService, UsersApiService

Contratos externos utilizados: SystemRole, UserResponse

8. Shared

Shared significa compartido. Aquí se guardan elementos que pueden usar distintas secciones del sistema, como el espacio donde se muestran los avisos y las estructuras de datos de acceso. Los archivos terminados en .spec.ts sirven para pruebas y no se cuentan como funciones independientes de la aplicación.

Clasificación	Nombre	Propósito	Ubicación
Component	ToastContainer	Sirve como espacio visual donde aparecen los avisos breves del sistema. Incluye un botón para cerrar cada aviso.	src/app/shared/components/toast-container/toast-container.ts
Type	SystemRole	Define los tres tipos de acceso reconocidos: administrador, operador y visualizador.	src/app/shared/models/api/auth.models.ts
Interface	LoginRequest	Define los datos que se envían al intentar iniciar sesión: usuario o correo y contraseña.	src/app/shared/models/api/auth.models.ts
Interface	UserResponse	Define la información de una cuenta recibida del servidor: identidad, correo, tipo de acceso, estado y fechas.	src/app/shared/models/api/auth.models.ts
Interface	LoginResponse	Define la respuesta al iniciar sesión: comprobante de acceso, fecha de vencimiento y datos del usuario.	src/app/shared/models/api/auth.models.ts
Interface	RegisterRequest	Define los datos que se enviarían para registrar una cuenta: nombre de usuario, correo y contraseña. Actualmente no hay una pantalla de registro.	src/app/shared/models/api/auth.models.ts
Interface	ProblemDetails	Define los datos con los que el servidor puede explicar un error, incluyendo su mensaje y los campos que necesitan corregirse.	src/app/shared/models/api/problem-details.model.ts

9. Models e Interfaces

Los modelos e interfaces son como fichas que indican qué datos debe tener algo. Por ejemplo, la ficha de un sensor define su nombre, ubicación y tipo de medición. Interface describe una estructura y type puede definir opciones permitidas. Sirven para organizar el código; no son pantallas ni realizan consultas por sí solos. En este proyecto no hay enums propios.

src/app/core/auth

Nombre	Tipo	Propósito	Ruta	Dónde se utiliza
AuthSession	Interface	Define la información que se guarda de una sesión: comprobante de acceso, fecha de vencimiento y datos de la persona conectada.	src/app/core/auth/auth-session.model.ts	src/app/core/auth/auth.store.ts; src/app/core/auth/to-ken-storage.service.ts

src/app/core/config

Nombre	Tipo	Propósito	Ruta	Dónde se utiliza
RealtimeConfig	Interface	Define las opciones para activar las actualizaciones en vivo y la dirección a la que debe conectarse la aplicación para recibirlas.	src/app/core/config/app-config.model.ts	Sin consumidor externo identificado en el código de producción; las referencias internas se indican en el propósito.
AppConfig	Interface	Define qué datos forman la configuración general:	src/app/core/config/app-	src/app/core/config/app-

Nombre	Tipo	Propósito	Ruta	Dónde se utiliza
		modo de ejecución, dirección del servidor y opciones de actualización en vivo.	config.model.ts	config.token.ts

src/app/core/guards

Nombre	Tipo	Propósito	Ruta	Dónde se utiliza
PendingChangesAware	Interface	Indica que una página debe poder responder si tiene cambios pendientes. El guard de salida necesita esa respuesta para decidir si permite abandonarla.	src/app/core/guards/pending-changes.guard.ts	Sin consumidor externo identificado en el código de producción; las referencias internas se indican en el propósito.

src/app/core/layout

Nombre	Tipo	Propósito	Ruta	Dónde se utiliza
NavIconName	Type	Define los nombres de los dibujos que se pueden usar en el menú de navegación.	src/app/core/layout/navigation.model.ts	src/app/core/layout/nav-icon/nav-icon.ts
NavItem	Interface	Define qué información necesita una opción del menú: texto visible, dirección a la que lleva e icono.	src/app/core/layout/navigation.model.ts	Sin consumidor externo identificado en el código de producción; las referencias internas se indican en el propósito.

src/app/core/notifications

Nombre	Tipo	Propósito	Ruta	Dónde se utiliza
ToastKind	Type	Define si un aviso es de éxito, error o información, para que se pueda presentar de forma diferente.	src/app/core/notifications/toast.service.ts	Sin consumidor externo identificado en el código de producción; las referencias internas se indican en el propósito.
ToastMessage	Interface	Define la información de cada aviso: número que lo identifica, tipo de aviso y texto que se muestra.	src/app/core/notifications/toast.service.ts	Sin consumidor externo identificado en el código de producción; las referencias internas se indican en el propósito.

src/app/core/realtime

Nombre	Tipo	Propósito	Ruta	Dónde se utiliza
RealtimeState	Type	Define las situaciones posibles de la conexión en vivo: desactivada, conectando, conectada, intentando reconectar o desconectada.	src/app/core/realtime/realtime.service.ts	Sin consumidor externo identificado en el código de producción; las referencias internas se indican en el propósito.
SensorStatusChanged	Interface	Define el aviso que indica qué sensor cambió de estado y si ahora está activo o inactivo.	src/app/core/realtime/realtime.service.ts	Sin consumidor externo identificado en el código de producción; las referencias internas se indican en el propósito.
NegotiateResponse	Interface	Define la respuesta que permite preparar la conexión en vivo con el servidor antes de empezar a recibir avisos.	src/app/core/realtime/realtime.service.ts	Sin consumidor externo identificado en el código de producción; las referencias internas se indican en el propósito.
HubMessage	Interface	Define la información de un mensaje recibido por la conexión en vivo, para reconocer qué ocurrió y qué datos llegaron.	src/app/core/realtime/realtime.service.ts	Sin consumidor externo identificado en el código de producción; las referencias internas se indican en el propósito.

src/app/features/alerts/models

Nombre	Tipo	Propósito	Ruta	Dónde se utiliza
AlertLevel	Type	Define los cuatro niveles de una alerta: verde, amarillo, naranja y rojo.	src/app/features/alerts/models/alert.models.ts	src/app/features/alerts/pages/alerts-page/alerts-page.ts; src/app/features/events/models/event.mod

Nombre	Tipo	Propósito	Ruta	Dónde se utiliza
				els.ts; src/app/features/events/pages/events-page/events-page.ts
RiskType	Type	Define los riesgos que reconoce el sistema: inundación, sequía, tormenta, helada e incendio forestal.	src/app/features/alerts/models/alert.models.ts	src/app/features/alerts/pages/alerts-page/alerts-page.ts; src/app/features/events/models/event.models.ts; src/app/features/events/pages/events-page/events-page.ts
AlertResponse	Interface	Define la información de una alerta recibida del servidor, como su riesgo, nivel, valores, estado, fechas y sensor y comunidad relacionados.	src/app/features/alerts/models/alert.models.ts	src/app/core/realtime/realtime.service.ts; src/app/features/alerts/components/alert-card/alert-card.ts; src/app/features/alerts/pages/alert-detail-page/alert-detail-page.ts; src/app/features/alerts/pages/alerts-page/alerts-page.ts; src/app/features/alerts/services/alerts-api.service.ts; src/app/features/dashboard/components/active-alerts-panel/active-alerts-panel.ts; src/app/features/dashboard/store/dashboard.store.ts; src/app/features/monitoring/store/monitoring.store.ts
AlertFilters	Interface	Define las opciones que se pueden enviar para buscar alertas, como nivel, riesgo, sensor, comunidad o estado.	src/app/features/alerts/models/alert.models.ts	src/app/features/alerts/pages/alerts-page/alerts-page.ts; src/app/features/alerts/services/alerts-api.service.ts

src/app/features/audit/models

Nombre	Tipo	Propósito	Ruta	Dónde se utiliza
AuditResponse	Interface	Define los datos de una acción registrada: quién la realizó, qué hizo, sobre qué recurso, desde qué dirección de red y cuándo ocurrió.	src/app/features/audit/models/audit.models.ts	src/app/features/audit/pages/audit-detail-page/audit-detail-page.ts; src/app/features/audit/pages/audit-page/audit-page.ts; src/app/features/audit/services/audit-api.service.ts
AuditFilters	Interface	Define las opciones para buscar acciones registradas por usuario, acción, recurso y fechas.	src/app/features/audit/models/audit.models.ts	src/app/features/audit/pages/audit-page/audit-page.ts; src/app/features/audit/services/audit-api.service.ts

src/app/features/communities/models

Nombre	Tipo	Propósito	Ruta	Dónde se utiliza
CommunityResponse	Interface	Define la información de una comunidad recibida del servidor: nombre, descripción, ubicación, estado y fecha de creación.	src/app/features/communities/models/community.models.ts	src/app/features/alerts/pages/alert-detail-page/alert-detail-page.ts; src/app/features/alerts/pages/alerts-page/alerts-page.ts

Nombre	Tipo	Propósito	Ruta	Dónde se utiliza
				page/alerts-page.ts; src/app/features/communities/pages/communities-page/communities-page.ts; src/app/features/communities/pages/community-detail-page/community-detail-page.ts; src/app/features/communities/services/communities-api.service.ts; src/app/features/events/pages/event-detail-page/event-detail-page.ts; src/app/features/events/pages/events-page/events-page.ts; src/app/features/sensors/pages/sensor-form-page/sensor-form-page.ts
CreateCommunityRequest	Interface	Define los datos que se envían para registrar una comunidad nueva: nombre, descripción y ubicación.	src/app/features/communities/models/community.models.ts	src/app/features/communities/pages/community-form-page/community-form-page.ts; src/app/features/communities/services/communities-api.service.ts
UpdateCommunityRequest	Interface	Define los datos que se envían para modificar una comunidad, incluyendo si está activa o inactiva.	src/app/features/communities/models/community.models.ts	src/app/features/communities/pages/community-form-page/community-form-page.ts; src/app/features/communities/services/communities-api.service.ts

src/app/features/dashboard/store

Nombre	Tipo	Propósito	Ruta	Dónde se utiliza
ClimateMetric	Interface	Sirve para asociar una variable climática con su lectura más reciente, si existe, y mostrarla en el dashboard.	src/app/features/dashboard/store/dashboard.store.ts	Sin consumidor externo identificado en el código de producción; las referencias internas se indican en el propósito.

src/app/features/events/models

Nombre	Tipo	Propósito	Ruta	Dónde se utiliza
EventResponse	Interface	Define la información de un evento climático recibido del servidor, incluyendo riesgo, nivel, fechas y sus relaciones con alerta, sensor y comunidad.	src/app/features/events/models/event.models.ts	src/app/features/events/pages/event-detail-page/event-detail-page.ts; src/app/features/events/pages/events-page/events-page.ts; src/app/features/events/services/events-api.service.ts
EventFilters	Interface	Define las opciones para buscar eventos por riesgo, nivel, sensor, comunidad y fechas.	src/app/features/events/models/event.models.ts	src/app/features/events/pages/events-page/events-page.ts; src/app/features/events/services/events-api.service.ts

src/app/features/monitoring/models

Nombre	Tipo	Propósito	Ruta	Dónde se utiliza
SensorType	Type	Define qué puede medir un sensor: temperatura, humedad, velocidad del viento, lluvia o nivel del agua.	src/app/features/monitoring/models/monitoring.models.ts	src/app/features/dashboard/components/climate-metric-card/climate-metric-card.ts; src/app/features/dashboard/store/dashboard.store.ts; src/app/features/sensors/models/sensor.models.ts; src/app/features/sensors/pages/sensor-form-page/sensor-form-page.ts
SensorReadingResponse	Interface	Define una medición recibida: qué sensor la produjo, qué mide, cuánto registró, en qué unidad y en qué momento.	src/app/features/monitoring/models/monitoring.models.ts	src/app/core/realtime/realtime.service.ts; src/app/features/dashboard/components/climate-metric-card/climate-metric-card.ts; src/app/features/dashboard/store/dashboard.store.ts; src/app/features/monitoring/services/monitoring-api.service.ts; src/app/features/monitoring/store/monitoring.store.ts
SimulationStatusResponse	Interface	Define la respuesta que indica si la simulación está funcionando o detenida.	src/app/features/monitoring/models/monitoring.models.ts	src/app/core/realtime/realtime.service.ts; src/app/features/dashboard/store/dashboard.store.ts; src/app/features/monitoring/components/simulation-control/simulation-control.ts; src/app/features/monitoring/services/monitoring-api.service.ts; src/app/features/monitoring/store/monitoring.store.ts
ChartPoint	Interface	Define un punto de una gráfica: el momento de la medición y el valor registrado.	src/app/features/monitoring/models/monitoring.models.ts	Sin consumidor externo identificado en el código de producción; las referencias internas se indican en el propósito.
SensorChartResponse	Interface	Define los datos que necesita la gráfica de un sensor: sensor, unidad y lista de puntos, que puede no contener información.	src/app/features/monitoring/models/monitoring.models.ts	src/app/features/monitoring/components/sensor-chart/sensor-chart.ts; src/app/features/monitoring/services/monitoring-api.service.ts; src/app/features/monitoring/store/monitoring.store.ts
ReadingHistoryFilters	Interface	Define las opciones para consultar lecturas anteriores por comunidad y rango de fechas.	src/app/features/monitoring/models/monitoring.models.ts	src/app/features/monitoring/services/monitoring-api.service.ts; src/app/features/monitoring/store/monitoring.store.ts

Nombre	Tipo	Propósito	Ruta	Dónde se utiliza
ChartFilters	Interface	Define las fechas y el intervalo de agrupación que se piden para una gráfica.	src/app/features/monitoring/models/monitoring.models.ts	src/app/features/monitoring/services/monitoring-api.service.ts; src/app/features/monitoring/store/monitoring.store.ts
CreateReadingRequest	Interface	Define los datos para enviar una medición nueva: sensor, valor y, si se proporciona, momento de la lectura.	src/app/features/monitoring/models/monitoring.models.ts	src/app/features/monitoring/services/monitoring-api.service.ts

src/app/features/sensors/models

Nombre	Tipo	Propósito	Ruta	Dónde se utiliza
SensorResponse	Interface	Define la información de un sensor recibida del servidor: identidad, tipo, unidad, comunidad, ubicación, estado y fechas.	src/app/features/sensors/models/sensor.models.ts	src/app/features/alerts/pages/alert-detail-page/alert-detail-page.ts; src/app/features/alerts/pages/alerts-page/alerts-page.ts; src/app/features/dashboard/store/dashboard.store.ts; src/app/features/events/pages/event-detail-page/event-detail-page.ts; src/app/features/events/pages/events-page/events-page.ts; src/app/features/monitoring/store/monitoring.store.ts; src/app/features/sensors/pages/sensor-detail-page/sensor-detail-page.ts; src/app/features/sensors/pages/sensors-page/sensors-page.ts; src/app/features/sensors/services/sensors-api.service.ts
SensorRequest	Interface	Define los datos que se envían para crear o modificar un sensor.	src/app/features/sensors/models/sensor.models.ts	src/app/features/sensors/pages/sensor-form-page/sensor-form-page.ts; src/app/features/sensors/services/sensors-api.service.ts

src/app/features/users/models

Nombre	Tipo	Propósito	Ruta	Dónde se utiliza
UpdateUserRequest	Interface	Define los datos que se envían para cambiar el nombre de usuario, correo y tipo de acceso de una cuenta.	src/app/features/users/models/user.models.ts	src/app/features/users/services/users-api.service.ts
UpdateUserStatusRequest	Interface	Define el dato que se envía para indicar si una cuenta debe quedar activa o inactiva.	src/app/features/users/models/user.models.ts	src/app/features/users/services/users-api.service.ts

src/app/shared/models/api

Nombre	Tipo	Propósito	Ruta	Dónde se utiliza
SystemRole	Type	Define los tres tipos de acceso reconocidos: administrador, operador y visualizador.	src/app/shared/models/api/auth.models.ts	src/app/core/auth/auth.store.ts; src/app/core/guards/role.guard.ts; src/app/features/use

Nombre	Tipo	Propósito	Ruta	Dónde se utiliza
				rs/models/user.model.s.ts; src/app/features/users/pages/user-detail-page/user-detail-page.ts; src/app/features/users/pages/user-edit-page/user-edit-page.ts; src/app/features/users/pages/users-page/users-page.ts
LoginRequest	Interface	Define los datos que se envían al intentar iniciar sesión: usuario o correo y contraseña.	src/app/shared/models/api/auth.models.ts	src/app/core/auth/auth-api.service.ts; src/app/core/auth/auth.service.ts
UserResponse	Interface	Define la información de una cuenta recibida del servidor: identidad, correo, tipo de acceso, estado y fechas.	src/app/shared/models/api/auth.models.ts	src/app/core/auth/auth-api.service.ts; src/app/core/auth/auth-session.model.ts; src/app/core/auth/auth.service.ts; src/app/core/auth/auth.store.ts; src/app/features/audit/pages/audit-page/audit-page.ts; src/app/features/users/pages/user-detail-page/user-detail-page.ts; src/app/features/users/pages/users-page/users-page.ts; src/app/features/users/services/users-api.service.ts
LoginResponse	Interface	Define la respuesta al iniciar sesión: comprobante de acceso, fecha de vencimiento y datos del usuario.	src/app/shared/models/api/auth.models.ts	src/app/core/auth/auth-api.service.ts
RegisterRequest	Interface	Define los datos que se enviarían para registrar una cuenta: nombre de usuario, correo y contraseña. Actualmente no hay una pantalla de registro.	src/app/shared/models/api/auth.models.ts	src/app/core/auth/auth-api.service.ts
ProblemDetails	Interface	Define los datos con los que el servidor puede explicar un error, incluyendo su mensaje y los campos que necesitan corregirse.	src/app/shared/models/api/problem-details.model.ts	src/app/core/http/problem-details.service.ts

10. Pipes

Un pipe cambia cómo se muestra un dato sin cambiar su significado. Por ejemplo, puede convertir una fecha en un texto como 09/09/2026 o mostrar un número con dos decimales. No hay pipes creados por el proyecto: se usan DatePipe para fechas y DecimalPipe para números, incluidos en Angular.

Pipe de Angular	Entrada → salida	Archivos consumidores
DatePipe	Fecha/string/número → texto de fecha	src/app/features/alerts/components/alert-card/alert-card.ts; src/app/features/alerts/pages/alert-detail-page/alert-detail-page.ts; src/app/features/audit/pages/audit-detail-page/audit-detail-page.ts; src/app/features/audit/pages/audit-page/audit-page.ts;

Pipe de Angular	Entrada → salida	Archivos consumidores
		src/app/features/communities/pages/communities-page/communities-page.ts; src/app/features/communities/pages/community-detail-page/community-detail-page.ts; src/app/features/dashboard/components/active-alerts-panel/active-alerts-panel.ts; src/app/features/dashboard/components/climate-metric-card/climate-metric-card.ts; src/app/features/dashboard/pages/dashboard-page/dashboard-page.ts; src/app/features/events/pages/event-detail-page/event-detail-page.ts; src/app/features/events/pages/events-page/events-page.ts; src/app/features/monitoring/pages/monitoring-page/monitoring-page.ts; src/app/features/sensors/pages/sensor-detail-page/sensor-detail-page.ts; src/app/features/sensors/pages/sensors-page/sensors-page.ts; src/app/features/users/pages/user-detail-page/user-detail-page.ts; src/app/features/users/pages/users-page/users-page.ts
DecimalPipe	Número/string numérico → texto decimal	src/app/features/alerts/components/alert-card/alert-card.ts; src/app/features/alerts/pages/alert-detail-page/alert-detail-page.ts; src/app/features/communities/pages/communities-page/communities-page.ts; src/app/features/communities/pages/community-detail-page/community-detail-page.ts; src/app/features/dashboard/components/climate-metric-card/climate-metric-card.ts; src/app/features/monitoring/pages/monitoring-page/monitoring-page.ts; src/app/features/sensors/pages/sensor-detail-page/sensor-detail-page.ts

11. Forms

Un formulario permite escribir o seleccionar datos. En los formularios reactivos, los campos y reglas se preparan en TypeScript con FormGroup y FormControl. En los formularios guiados por la plantilla se conectan desde HTML con ngModel. Hay nueve páginas con formularios reactivos y dos con controles ngModel. No se usa FormBuilder.

Componente / ubicación	Tipo	Propósito	Validaciones
LoginPage — src/app/features/auth/pages/login-page/login-page.ts	Reactivo	Inicio de sesión	login requerido, máximo 256; password requerido, mínimo 8 y máximo 256.
SensorFormPage — src/app/features/sensors/pages/sensor-form-page/sensor-form-page.ts	Reactivo	Crear o editar sensor	name requerido/máx. 120; code requerido/máx. 50; description máx. 500; type, unit y communityId requeridos; unit máx. 20; latitud requerida entre -90 y 90; longitud requerida entre -180 y 180.
CommunityFormPage — src/app/features/communities/pages/community-form-page/community-form-page.ts	Reactivo	Crear o editar comunidad	name requerido/máx. 120; description máx. 500; latitud requerida entre -90 y 90; longitud requerida entre -180 y 180; isActive sin Validators.
UserEditPage —	Reactivo	Editar identidad y rol	username requerido/máx. 100;

Componente / ubicación	Tipo	Propósito	Validaciones
<code>src/app/features/users/pages/user-edit-page/user-edit-page.ts</code>			email requerido, formato email y máx. 254; role requerido.
<code>AlertsPage — src/app/features/alerts/pages/alerts-page/alerts-page.ts</code>	Reactivo	Filtrar riesgo, nivel, sensor, comunidad y estado	Sin Validators declarados; valores vacíos se convierten a undefined en filtros.
<code>EventsPage — src/app/features/events/pages/events-page/events-page.ts</code>	Reactivo	Filtrar historial y fechas	Sin ValidatorFn; applyFilters rechaza from posterior a to cuando ambas fechas están presentes.
<code>AuditPage — src/app/features/audit/pages/audit-page/audit-page.ts</code>	Reactivo	Filtrar bitácora y fechas	Sin ValidatorFn; applyFilters comprueba from <= to.
<code>MonitoringPage — src/app/features/monitoring/pages/monitoring-page/monitoring-page.ts</code>	Reactivo	Rango de lectura e intervalo	Sin Validators ni comparación de orden de fechas en applyFilters; convierte fechas no vacías a ISO.
<code>UsersPage — src/app/features/users/pages/users-page/users-page.ts</code>	Reactivo	Búsqueda, rol y estado local	Sin Validators declarados.
<code>SensorsPage — src/app/features/sensors/pages/sensors-page/sensor-s-page.ts</code>	Template-driven, control ngModel	Búsqueda local	Sin Validators declarados; query se normaliza al comparar.
<code>CommunitiesPage — src/app/features/communities/pages/communities-page/communities-page.ts</code>	Template-driven, control ngModel	Búsqueda local	Sin Validators declarados; query se normaliza al comparar.

12. Stores y gestión de estado

Un store es una parte del programa que guarda y organiza información que necesita una pantalla o varias partes de la aplicación. Por ejemplo, el store del dashboard reúne las mediciones y los recuentos de sensores. Los tres stores del proyecto están creados con herramientas de Angular y RxJS; no utilizan una biblioteca adicional de estado.

AuthStore

Categoría: Store

Descripción: Sirve para mantener a mano la información de la persona que inició sesión y saber qué acciones puede realizar. También recupera una sesión guardada si cumple las condiciones de acceso del sistema.

Ruta del archivo:

`src/app/core/auth/auth.store.ts`

Utilizado por: `src/app/app.ts`; `src/app/core/auth/auth.service.ts`; `src/app/core/guards/auth.guard.ts`; `src/app/core/guards/guest.guard.ts`; `src/app/core/guards/role.guard.ts`; `src/app/core/interceptors/api-error.interceptor.ts`; `src/app/core/interceptors/auth.interceptor.ts`; `src/app/core/layout/sidebar/sidebar.ts`; `src/app/core/layout/topbar/topbar.ts`; `src/app/core/realtime/realtime.service.ts`; `src/app/features/alerts/pages/alert-detail-page/alert-detail-page.ts`; `src/app/features/alerts/pages/alerts-page/alerts-page.ts`; `src/app/features/communities/pages/communities-page/communities-page.ts`; `src/app/features/communities/pages/community-detail-page/community-detail-page.ts`; `src/app/features/monitoring/pages/monitoring-page/monitoring-page.ts`; `src/app/features/sensors/pages/sensor-detail-page/sensor-detail-page.ts`; `src/app/features/sensors/pages/sensors-page/sensors-page.ts`; `src/app/features/users/pages/user-edit-page/user-edit-page.ts`; `src/app/features/users/pages/users-page/users-page.ts`

Dependencias principales: `TokenStorageService`

Ámbito: Global; providedIn: root.

Métodos principales: `restoreSession`, `setSession`, `updateUser`, `clearSession`, `hasAnyRole`

DashboardStore

Categoría: Store

Descripción: Sirve para reunir y mantener actualizados los datos que necesita el dashboard: mediciones, sensores, alertas y estado de la simulación. También calcula los resúmenes que se muestran en esa pantalla.

Ruta del archivo:

`src/app/features/dashboard/store/dashboard.store.ts`

Utilizado por: `src/app/features/dashboard/pages/dashboard-page/dashboard-page.ts`

Dependencias principales: `MonitoringApiService`, `SensorsApiService`, `AlertsApiService`, `RealtimeService`, `DestroyRef`

Ámbito: Instancia local mediante providers de `DashboardPage`.

Métodos principales: `load`

MonitoringStore

Categoría: Store

Descripción: Sirve para organizar los datos de la pantalla de monitoreo: sensor seleccionado, última lectura, historial, gráfica y simulación. Recibe cambios en vivo y, si esa conexión falla y la pestaña está visible, consulta datos generales cada cinco segundos.

Ruta del archivo:

`src/app/features/monitoring/store/monitoring.store.ts`

Utilizado por: `src/app/features/monitoring/pages/monitoring-page/monitoring-page.ts`

Dependencias principales: `MonitoringApiService`, `SensorsApiService`, `AlertsApiService`, `ToastService`, `DOCUMENT`, `DestroyRef`, `RealtimeService`

Ámbito: Instancia local mediante providers de `MonitoringPage`.

Métodos principales: `initialize`, `selectSensor`, `loadDetails`, `startSimulation`, `stopSimulation`, `resetSimulation`

13. Components — páginas principales

Esta sección presenta solo las páginas principales que se abren desde una dirección del navegador. Cada ficha indica qué contiene la página, para qué sirve y dónde está su archivo. Los elementos pequeños de la pantalla, como menús y tarjetas, no se incluyen como páginas principales.

No existe una página de «Crear cuenta» en el frontend actual. Hay un método de registro en `AuthApiService`, pero no un componente ni una ruta de registro.

Iniciar sesión (LoginPage)

Categoría: Component

Descripción: Contiene los campos de usuario y contraseña y los mensajes de error. Sirve para acceder al sistema con una cuenta existente.

Ruta del archivo:

`src/app/features/auth/pages/login-page/login-page.ts`

Ruta de navegación: `/login`

Dashboard (DashboardPage)

Categoría: Component

Descripción: Contiene indicadores climáticos, recuentos de sensores, estado de la simulación y alertas recientes. Sirve para consultar un resumen del estado del sistema.

Ruta del archivo:

`src/app/features/dashboard/pages/dashboard-page/dashboard-page.ts`

Ruta de navegación: `/dashboard`

Monitoreo (MonitoringPage)

Categoría: Component

Descripción: Contiene selección de sensor, última lectura, filtros de fechas, gráfica, histórico, alertas y controles de simulación. Sirve para seguir las variables climáticas y operar la simulación según los permisos.

Ruta del archivo:

`src/app/features/monitoring/pages/monitoring-page/monitoring-page.ts`

Ruta de navegación: `/monitoring`

Listado de sensores (SensorsPage)

Categoría: Component

Descripción: Contiene la tabla de sensores, búsqueda y acciones de consulta, edición, activación y eliminación. Sirve para administrar el inventario de sensores.

Ruta del archivo:

`src/app/features/sensors/pages/sensors-page/sensors-page.ts`

Ruta de navegación: `/sensors`

Crear o editar sensor (SensorFormPage)

Categoría: Component

Descripción: Contiene el formulario de nombre, código, descripción, tipo, unidad, comunidad y coordenadas. Sirve para registrar un sensor o actualizar sus datos.

Ruta del archivo:

`src/app/features/sensors/pages/sensor-form-page/sensor-form-page.ts`

Ruta de navegación: `/sensors/new, /sensors/:id/edit`

Detalle del sensor (SensorDetailPage)

Categoría: Component

Descripción: Contiene los datos, ubicación, comunidad, estado y fechas de un sensor. Sirve para consultar su información individual.

Ruta del archivo:

`src/app/features/sensors/pages/sensor-detail-page/sensor-detail-page.ts`

Ruta de navegación: `/sensors/:id`

Listado de comunidades (CommunitiesPage)

Categoría: Component

Descripción: Contiene el listado de comunidades, búsqueda y enlaces de consulta y edición. Sirve para localizar y administrar las comunidades registradas.

Ruta del archivo:

src/app/features/communities/pages/communities-page/communities-page.ts

Ruta de navegación: /communities

Crear o editar comunidad (CommunityFormPage)

Categoría: Component

Descripción: Contiene el formulario de nombre, descripción y coordenadas, además del estado al editar. Sirve para registrar una comunidad o actualizar su información.

Ruta del archivo:

src/app/features/communities/pages/community-form-page/community-form-page.ts

Ruta de navegación: /communities/new, /communities/:id/edit

Detalle de comunidad (CommunityDetailPage)

Categoría: Component

Descripción: Contiene nombre, descripción, coordenadas, estado y fecha de creación. Sirve para consultar la información de una comunidad.

Ruta del archivo:

src/app/features/communities/pages/community-detail-page/community-detail-page.ts

Ruta de navegación: /communities/:id

Listado de alertas (AlertsPage)

Categoría: Component

Descripción: Contiene filtros de riesgo, nivel, sensor, comunidad y estado, junto con las alertas encontradas y su confirmación de resolución. Sirve para consultar y gestionar alertas climáticas.

Ruta del archivo:

src/app/features/alerts/pages/alerts-page/alerts-page.ts

Ruta de navegación: /alerts

Detalle de alerta (AlertDetailPage)

Categoría: Component

Descripción: Contiene riesgo, nivel, valores, fechas y contexto del sensor y la comunidad, además de la opción de resolver según permisos. Sirve para revisar y atender una alerta específica.

Ruta del archivo:

src/app/features/alerts/pages/alert-detail-page/alert-detail-page.ts

Ruta de navegación: /alerts/:id

Historial de eventos (EventsPage)

Categoría: Component

Descripción: Contiene filtros y un listado de eventos climáticos. Sirve para buscar sucesos por riesgo, nivel, sensor, comunidad y rango de fechas.

Ruta del archivo:

src/app/features/events/pages/events-page/events-page.ts

Ruta de navegación: /events

Detalle de evento (EventDetailPage)

Categoría: Component

Descripción: Contiene los datos del evento, sus fechas y el contexto del sensor y la comunidad. Sirve para consultar un suceso individual.

Ruta del archivo:

src/app/features/events/pages/event-detail-page/event-detail-page.ts

Ruta de navegación: /events/:id

Listado de usuarios (UsersPage)

Categoría: Component

Descripción: Contiene la tabla de usuarios, filtros de búsqueda, rol y estado, y acciones para editar o cambiar la activación. Sirve para administrar las cuentas registradas.

Ruta del archivo:

src/app/features/users/pages/users-page/users-page.ts

Ruta de navegación: /users

Editar usuario (UserEditPage)

Categoría: Component

Descripción: Contiene un formulario de nombre de usuario, correo electrónico y rol. Sirve para actualizar la identidad y el nivel de acceso de una cuenta.

Ruta del archivo:

src/app/features/users/pages/user-edit-page/user-edit-page.ts

Ruta de navegación: /users/:id/edit

Detalle de usuario (UserDetailPage)

Categoría: Component

Descripción: Contiene identidad, correo, rol, estado y fechas de la cuenta. Sirve para consultar la información de un usuario.

Ruta del archivo:

src/app/features/users/pages/user-detail-page/user-detail-page.ts

Ruta de navegación: /users/:id

Auditoría (AuditPage)

Categoría: Component

Descripción: Contiene filtros por usuario, acción, recurso y fechas, junto con la bitácora de actividad. Sirve para consultar las acciones registradas en el sistema.

Ruta del archivo:

`src/app/features/audit/pages/audit-page/audit-page.ts`

Ruta de navegación: `/audit`

Detalle de auditoría (AuditDetailPage)

Categoría: Component

Descripción: Contiene usuario, acción, recurso, descripción, dirección IP y fecha de un registro. Sirve para revisar una entrada específica de la bitácora.

Ruta del archivo:

`src/app/features/audit/pages/audit-detail-page/audit-detail-page.ts`

Ruta de navegación: `/audit/:id`

Sistema (FeaturePlaceholderPage)

Categoría: Component

Descripción: Contiene un título y un aviso de funcionalidad pendiente. Sirve como página provisional de la sección de configuración del sistema.

Ruta del archivo:

`src/app/core/layout/feature-placeholder-page/feature-placeholder-page.ts`

Ruta de navegación: `/system`

Acceso restringido (ForbiddenPage)

Categoría: Component

Descripción: Contiene un mensaje de falta de permisos y un enlace al dashboard. Sirve para informar que la cuenta no tiene acceso a la sección solicitada.

Ruta del archivo:

`src/app/core/layout/forbidden-page/forbidden-page.ts`

Ruta de navegación: `/forbidden`